

CS 550
Formal Verification
Midterm Exam

November 27, 2025

Name: Ada Lovelace
Sciper: 111111

Room: GR A3 30
Seat: 314

1. You have **165 minutes** to complete the exam.
2. Place your CAMIPRO card on your desk.
3. Put all electronic devices in a bag away from the bench.
4. Write your final answers using a permanent pen (no pencils, no erasable pens).
5. This exam is 19 pages long, including this cover page. Check that you have all the pages.
6. The exercises are not ordered by how difficult they may be. If you are stuck on an exercise, you can skip it and come back to it later.
7. Answer all questions in the provided space. Do not submit additional sheets unless you get explicit permission to do so. Do not unstaple the given sheets. Material in the scratch area will not be graded.
8. The maximum number of points on the exam is **40**.

Question:	1	2	3	4	5	6	7	Total
Points:	3	5	5	6	4	6	11	40
Score:								



Page intentionally left blank.

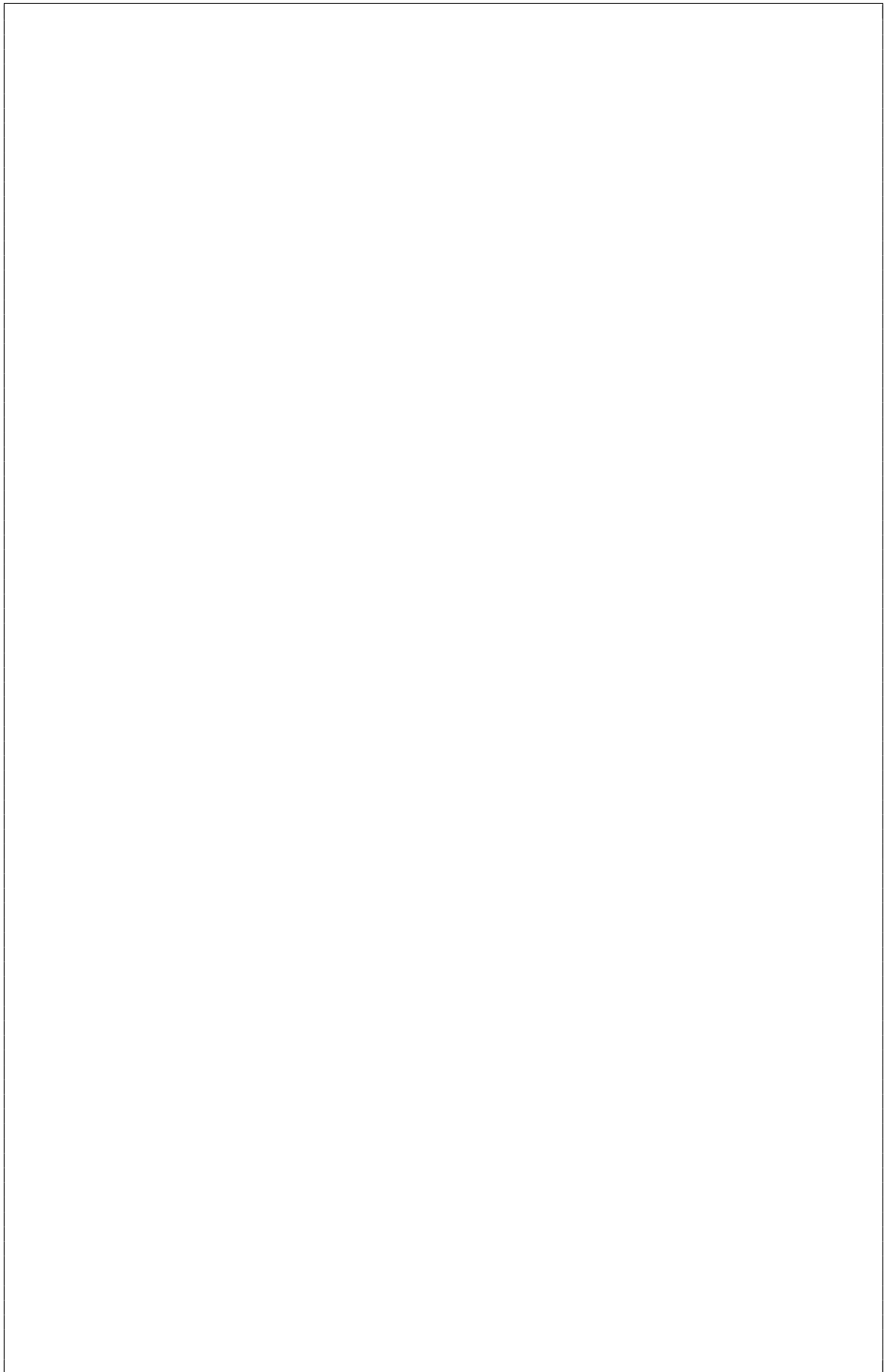
**Question 1.**

(3 points)

Consider the following propositional formula α :

$$((p \implies q) \wedge (p \implies \neg q)) \implies \neg p .$$

1. 1 point Transform α to an equivalent formula in negation-normal form that uses only $\wedge$, $\vee$, and $\neg$. Use a polynomial-time algorithm. Denote the result by ϕ .
2. 1 point Draw an abstract syntax tree for ϕ , with propositional connectives as the internal nodes. Label the binary nodes in your tree such that children have larger numbers than their parent, starting from 0 at the root (you can, for example use depth-first tree traversal).
3. 1 point Convert the formula into an equisatisfiable clausal normal form (CNF) by applying Tseitin's transformation. For variables you introduce for binary connectives, use x_n as the name, where n is the node number for the corresponding subformula in your syntax tree.



**Question 2.**

(5 points)

Consider a first-order theory with two binary relations, L (meant to denote some partial ordering relation $\sqsubseteq$) and E (meant to denote equality $=$). We define the shorthand $\text{lub}(x, y, z)$ as

$$L(x, z) \wedge L(y, z) \wedge \forall w. (L(x, w) \wedge L(y, w)) \implies L(z, w) .$$

Then, define the axiom A_{lub} as

$$\forall x, y. \exists z. \text{lub}(x, y, z) .$$

1. 1 point Show the result of transforming the axiom A_{lub} into an equisatisfiable finite set of clauses.
2. 4 points Given the set of numbered clausal form axioms below, and the axiom A_{lub} , prove the following statement by constructing a resolution refutation for its negation:

$$\phi := \forall x, y, z_1, z_2. (\text{lub}(x, y, z_1) \wedge \text{lub}(x, y, z_2)) \implies E(z_1, z_2) .$$

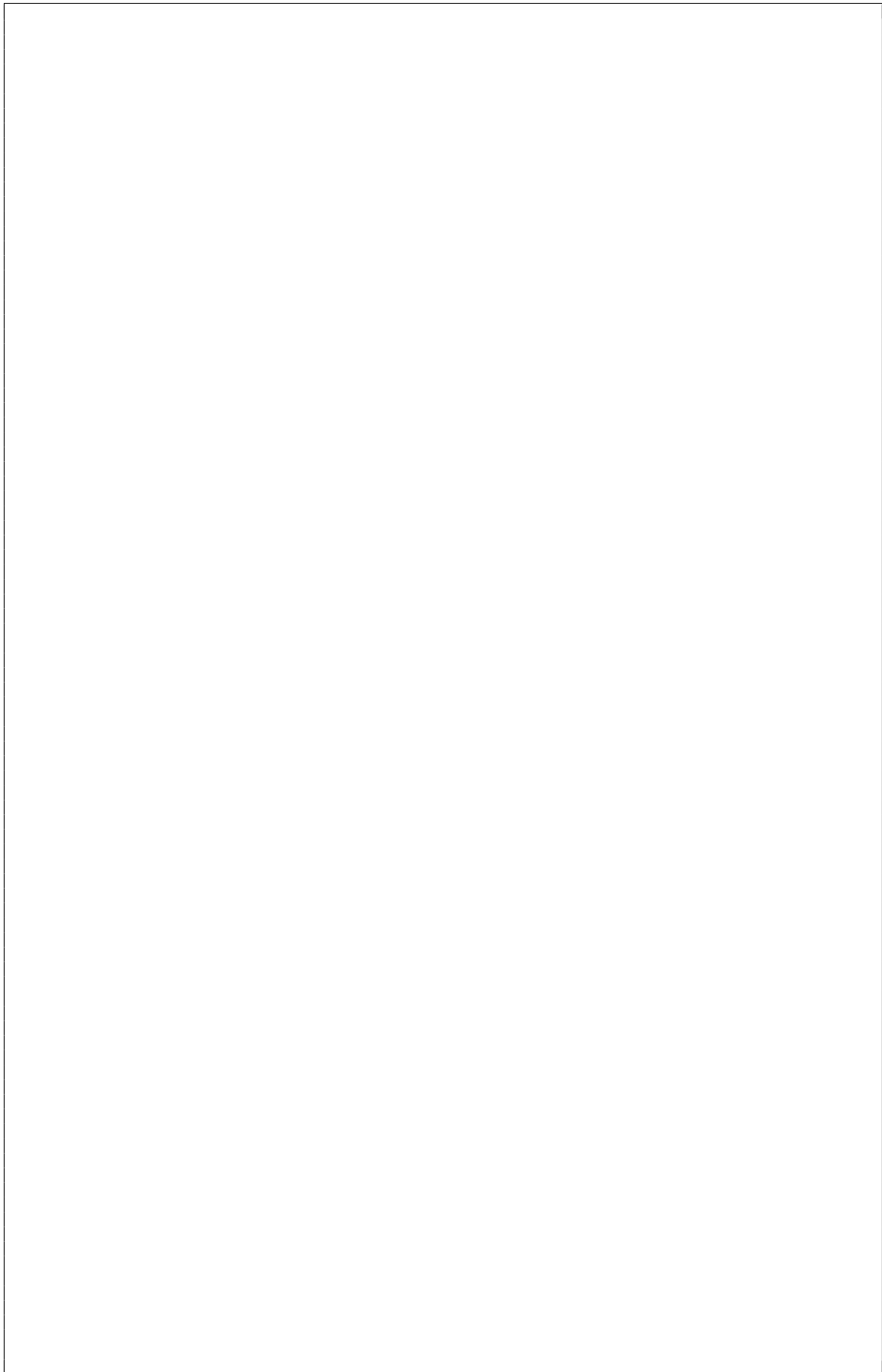
Number each step following the axioms and use them to refer to premises of each step in your proof.

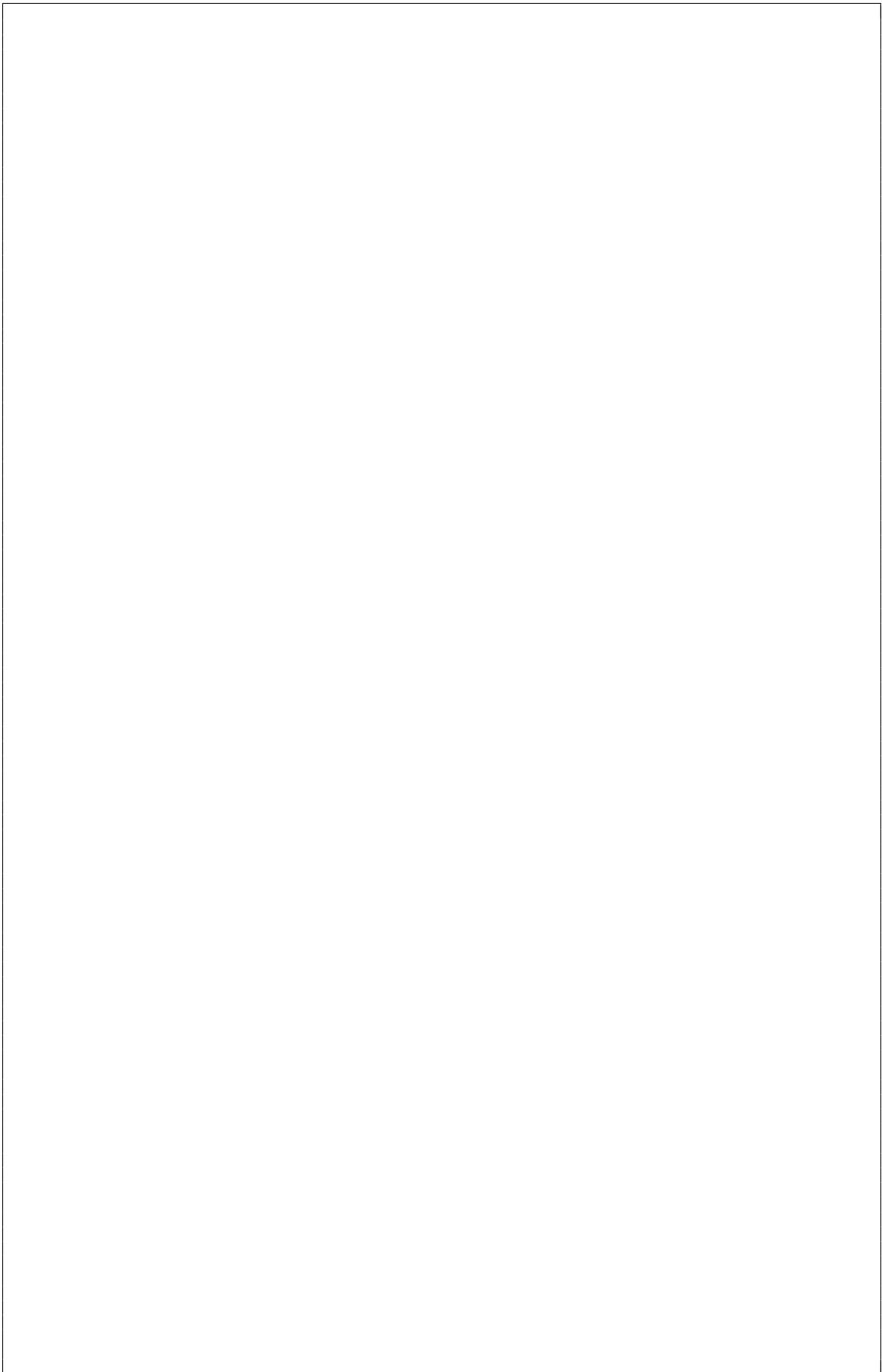
Note that $\text{lub}(x, y, z)$ is just shorthand, and you can expand it as needed.

Start your proof with the following axioms for equality E and partial order L :

- 1 : $E(x, x)$
- 2 : $\neg E(x, y) \vee \neg E(y, z) \vee E(x, z)$
- 3 : $E(x, y) \vee \neg E(y, x)$
- 4 : $L(x, x)$
- 5 : $\neg L(x, y) \vee \neg L(y, z) \vee L(x, z)$
- 6 : $\neg L(x, y) \vee \neg L(y, x) \vee E(x, y)$

where x, y , and z are free variables (implicitly universally quantified). Add clauses for A_{lub} and proceed with resolution steps.





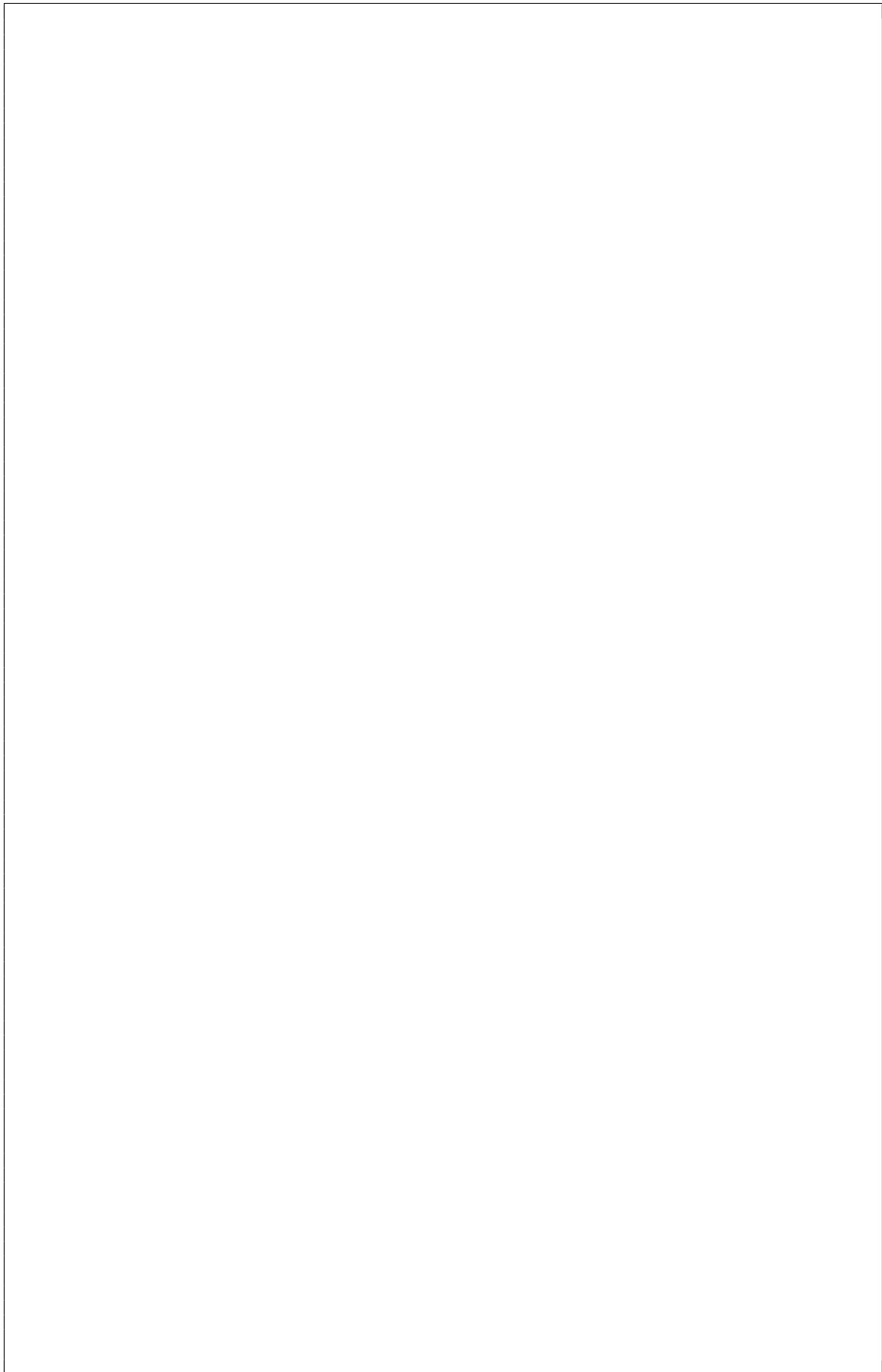
**Question 3.**

(5 points)

Consider the following Presburger arithmetic formula ϕ :

$$\forall x. \exists y. x < y \wedge y < x + 6 \wedge 3 \mid (y + 2)$$

Check whether the formula is true or false using quantifier elimination. Show your steps. You may use arbitrary equivalence-preserving transformations on quantifier-free subformulas in a single step.



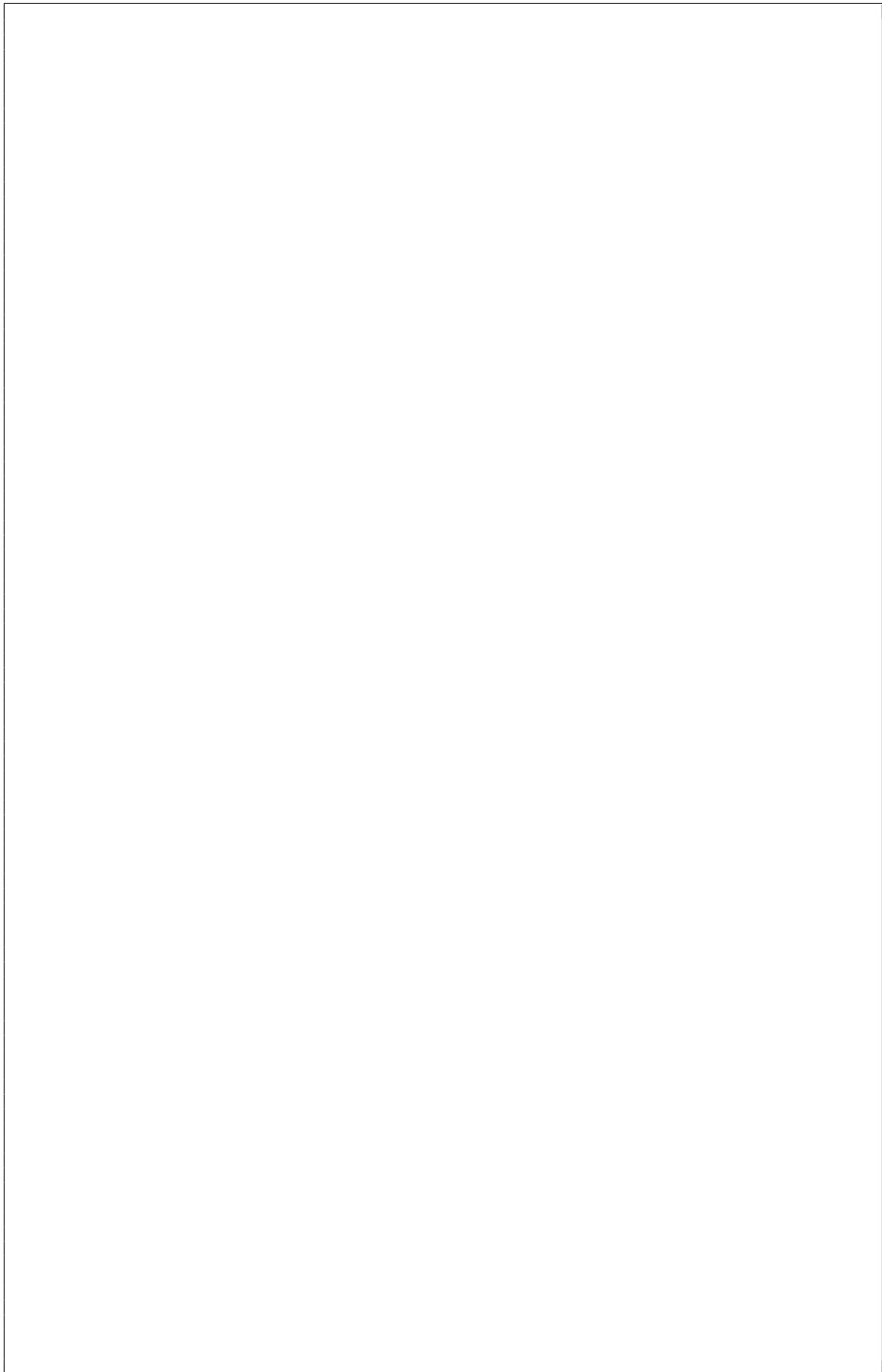
**Question 4.** (6 points)

Let $(C, \leq)$ and $(A, \sqsubseteq)$ be two partial orders. Let $F : C \rightarrow A$ and $G : A \rightarrow C$ be two functions such that (F, G) they satisfy the Galois condition:

$$F(c) \sqsubseteq a \iff c \leq G(a).$$

1. 2 points Prove that F and G are monotone functions, and thus that (F, G) is a Galois connection.
2. 4 points Suppose there exists another monotone function $H : A \rightarrow C$ such that (F, H) is also a Galois connection. Then, prove that $G(a) = H(a)$ for every $a \in A$.

For the second part, you may assume the results of the first.



**Question 5.**

(4 points)

Let $(A, \leq)$ be a partial order. For each $a \in A$, define the set L_a as

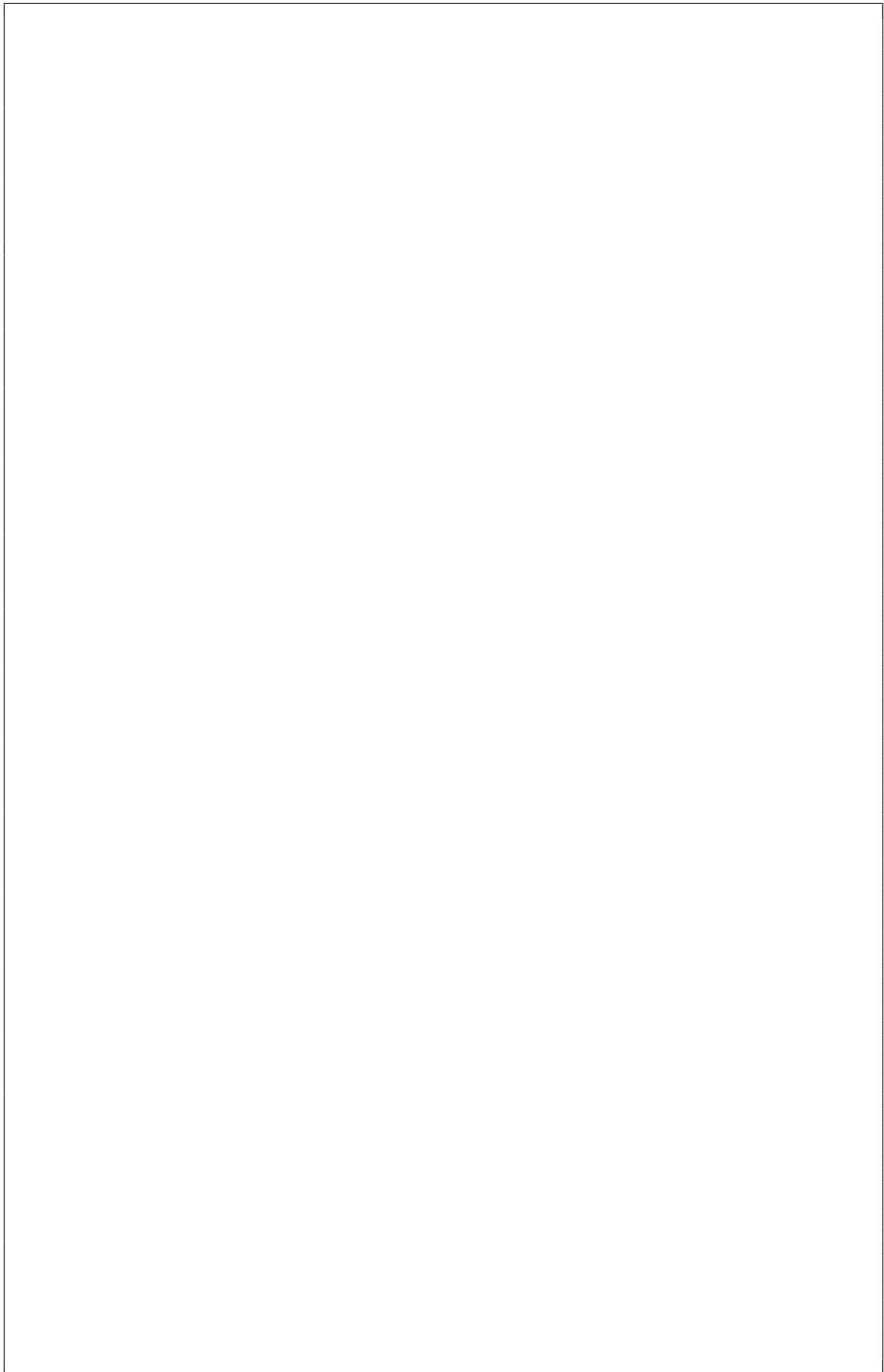
$$L_a = \{x \in A \mid x \leq a\}.$$

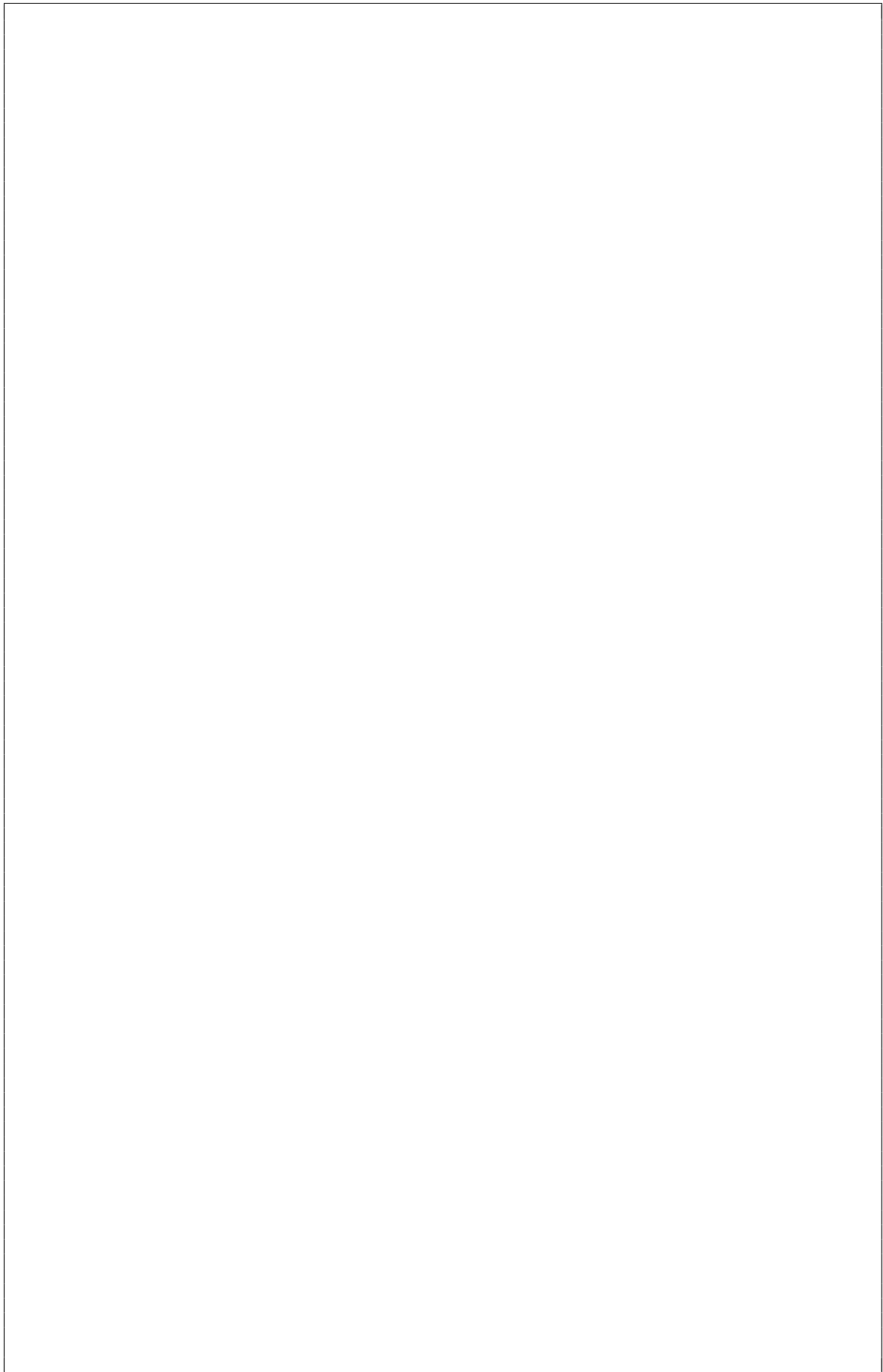
Let $C = \{L_a \mid a \in A\}$ be the collection of all such sets. Define the function $L : A \rightarrow C$ by $L(a) = L_a$.

Consider the partial order $(C, \subseteq)$, where $\subseteq$ is the usual subset relation. Answer each of the following questions with a proof or a counterexample:

1. 1 point Is L monotone from $(A, \leq)$ to $(C, \subseteq)$?
2. 1 point Is L injective?
3. 1 point Does there exist a function $K_1 : C \rightarrow A$ such that (L, K_1) is a Galois connection between $(A, \leq)$ and $(C, \subseteq)$?
4. 1 point Does there exist a function $K_2 : C \rightarrow A$ such that (L, K_2) is a Galois insertion of $(C, \subseteq)$ into $(A, \leq)$?

Recall that (L, K) is a Galois insertion of $(C, \subseteq)$ into $(A, \leq)$ if it is a Galois connection between them and for every $c \in C$, we have $L(K(c)) = c$.





**Question 6.**

(6 points)

Prove the following first-order logic sequent using Lisa proof steps:

$$\forall x. P(x) \implies S(x), \exists x. \neg S(x) \quad \vdash \quad \exists x. \neg P(x)$$

Exact syntax of proof steps is not important. You may begin with the following definitions and theorem scaffold.

Common proof steps you may find useful are described in the appendix.

```
val x = variable[Ind]
val P = variable[Ind >>: Prop]
val S = variable[Ind >>: Prop]

Theorem( ( ∀(x, P(x) ⇒ S(x)), ∃(x, !S(x)) ) ⊢ ∃(x, !P(x)) ):
  // your proof here
```

**Question 7.**

(11 points)

Consider the following program over mathematical (unbounded) integers (BigInts):

```
def myLittleProgram(): BigInt =
  var a: BigInt = havoc()
  var b: BigInt = havoc() ①

  assume(a ≥ 0 ∧ b ≥ 0) ①

  while ② (a != 0 ∧ b != 0) { ③
    if a ≥ b then ④
      a = a - b ⑤
    else ⑥
      b = b - a ⑦
  } ⑧

  assert(a ≥ 0 ∧ b ≥ 0) ⑨
  max(a, b)
```

We identify program points using numbers ①, ②, ③, Recall that `havoc()` non-deterministically returns an arbitrary integer value, `assume(p)` restricts the execution to states where p holds.

`assert(p)` checks that p holds in the state, or passes the control-flow to a distinct error state.

Consider the following set of predicates, labelled p_1, p_2, p_3 , and p_4 from left to right:

$$\mathcal{P} = \{ \underbrace{a \geq 0}_{p_1}, \underbrace{b \geq 0}_{p_2}, \underbrace{a \geq b}_{p_3}, \underbrace{a > 0}_{p_4} \}$$

1. 1 point Draw the Hasse diagram for the lattice $(2^{\mathcal{P}}, \sqsubseteq)$ of conjuncts over elements of $\mathcal{P}$ ordered by the predicate abstraction lattice order.



2. 2 points Using the numbered program points above as the set of states, draw a control-flow graph representing `myLittleProgram`. Annotate the edges with guards and assignments. Introduce a program point `err` to which the program moves when the assertion condition does not hold. There should be no outgoing edges from `err`. The set of states is $Q = \{0, 1, \dots, 9, \text{err}\}$.

A large, empty rectangular box with a thin black border, intended for drawing a control-flow graph.



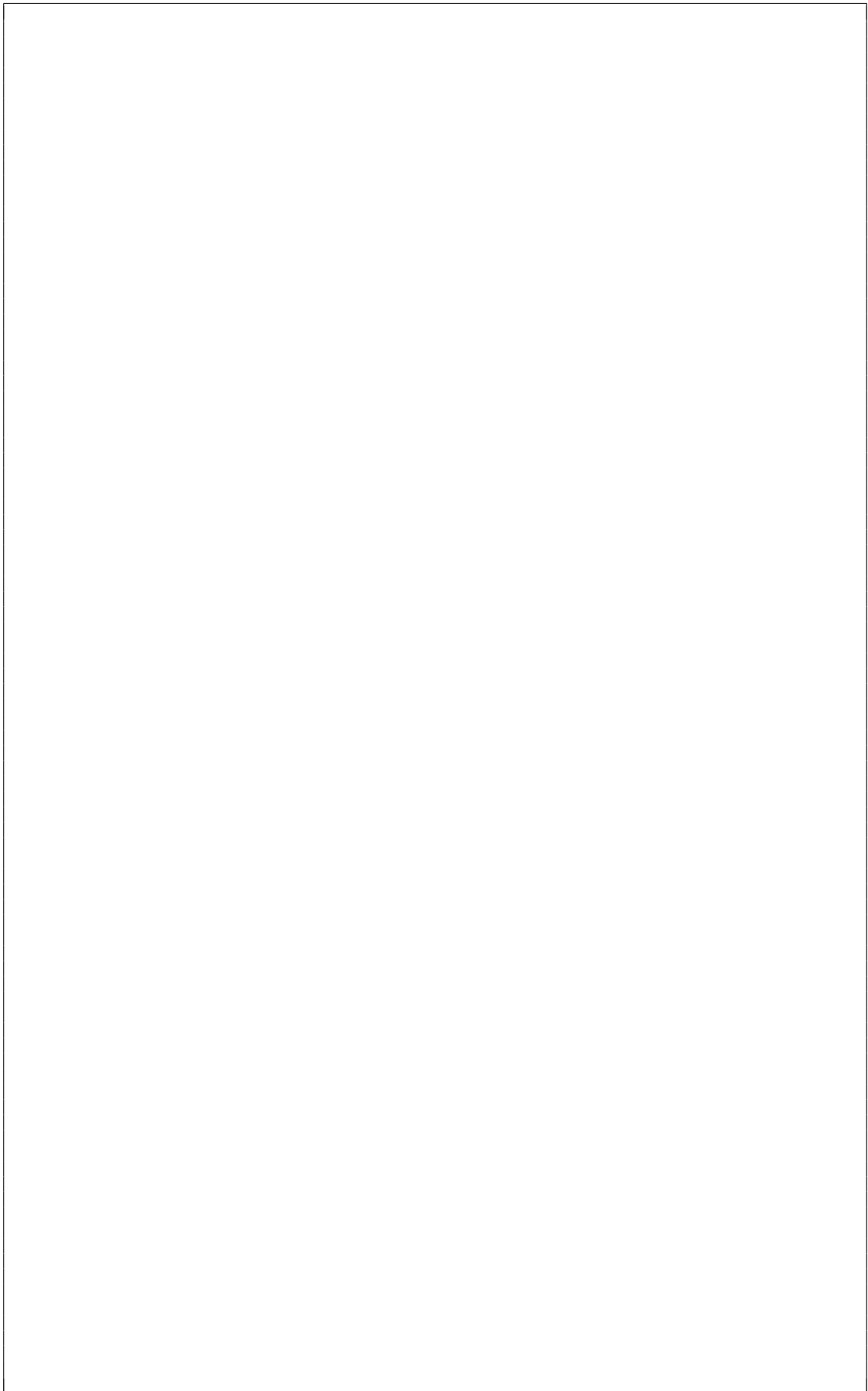
3. 2 points Represent the semantics of each edge in your transition system as a formula over pre- and post-state variables, (a, b) and (a', b') . Represent the edge from state $i \in Q$ to $j \in Q$ as the formula labelled $R_{i \rightarrow j}(a, b, a', b')$.

4. 1 point Denote the abstract set of values at each program point i by the function $S^\#(i)$ for $i \in \{0, 1, \dots, 9, \text{err}\}$. Further, denote the abstract transfer function by $T^\#(S, R)$. It takes $S \in 2^{\mathcal{P}}$, an abstract set of values, and R , a formula relating concrete states as in the previous part. It computes the new abstract set of values $\in 2^{\mathcal{P}}$.

Using $S^\#$ and $T^\#$, write a set of recursive equations modelling the transition system, whose least fixed point solution gives the abstract set of values at each program point.



5. 3 points Compute and show the least fixed point of the recursive equations computed in the previous part. You do not need to prove that the result is indeed a fixed point or the least.





6. 1 point Give a mathematical expression describing the set of possible concrete values of a and b corresponding to the computed fixed point at the program point `err`?
- 1 point Is there an execution of this program that reaches the `err` state? Justify your answer.



Appendix

Basic Lisa proof steps for quantifiers and propositional reasoning, with usage examples.

Tautology. Proves a propositional tautology.

$$\frac{\Gamma_1 \vdash \Phi_1 \quad \Gamma_2 \vdash \Phi_2 \quad \dots \quad \Gamma_n \vdash \Phi_n}{\Gamma \vdash \Psi} \text{Tautology}$$

such that $(\bigwedge_{i=1}^n (\Gamma_i \Rightarrow \Phi_i)) \Rightarrow (\Gamma \Rightarrow \Psi)$ is provable in propositional logic.

Usage example:

```
have(B ⊢ A ⇒ B) by Tautology
have((A, B) ⊢ B ⇒ A) by Tautology.from(lastStep)
```

InstantiateForall. Instantiate a universally quantified formula.

$$\frac{}{\Gamma, \forall x. P(x) \vdash P(t)} \text{InstantiateForall (1)}$$

$$\frac{\Gamma \vdash \forall x. P(x)}{\Gamma \vdash P(t)} \text{InstantiateForall (2)}$$

where t is any term.

Usage example:

```
// version 1
have((B, ∀(x, P(x))) ⊢ P(t)) by InstantiateForall

// version 2
have(B ⊢ ∀(x, P(x))) by ...
thenHave(B ⊢ P(y)) by InstantiateForall(y)
```

RightExists. Introduce an existential quantifier on the right.

$$\frac{\Gamma \vdash P(t)}{\Gamma \vdash \exists x. P(x)} \text{RightExists}$$

where t is any term.

Usage example:

```
have(B ⊢ P(t)) by ...
thenHave(B ⊢ ∃(x, P(x))) by RightExists
```

LeftExists. Introduce an existential quantifier on the left.

$$\frac{\Gamma, P(x) \vdash \Phi}{\Gamma, \exists x. P(x) \vdash \Phi} \text{LeftExists}$$

where x is *not free* in the resulting sequent.

Usage example:

```
have((B, P(y)) ⊢ Q) by ...
thenHave((B, ∃(x, P(x))) ⊢ Q) by LeftExists
```



RightForall. Introduce a universal quantifier on the right.

$$\frac{\Gamma \vdash P(x)}{\Gamma \vdash \forall x. P(x)} \text{RightForall}$$

where x is *not free* in the resulting sequent.

Usage example:

```
have(B ⊢ P(y)) by ...
thenHave(B ⊢ ∀(x, P(x))) by RightForall
```

LeftForall. Introduce a universal quantifier on the left.

$$\frac{\Gamma, P(t) \vdash \Phi}{\Gamma, \forall x. P(x) \vdash \Phi} \text{LeftForall}$$

where t is any term.

Usage example:

```
have((B, P(t)) ⊢ R(t)) by ...
thenHave((B, ∀(x, P(x))) ⊢ R(t)) by LeftForall
```



Scratch area

End